

Chapitre 2

Résolution numérique de systèmes linéaires

Notes rédigées par Raphaël JONTEF

cours dispensé par Marc Durufle

Enseirb-Matmeca
filière informatique

11 février 2026

Table des matières

I Résolution de systèmes linéaires

Soit $A \in \mathcal{M}_n(\mathbb{R})$, $B \in \mathbb{R}^n$. En pratique, la solution numérique $x + \delta x$ de l'égalité vectorielle $AX = B$ vérifie :

$$A \times (x + \delta x) = b + \delta b$$

On a alors :

$$\delta b = A \times (x + \delta x) - b$$

Le vecteur x étant inconnu, on ne peut pas calculer directement δb . On cherche donc à majorer $\|\delta b\|$.

I.1 Rappels sur les normes usuelles sur $\mathbb{R}^n$

Définition 2.1 - norme infinie

Pour $X \in \mathbb{R}^n$, on définit la norme infinie par :

$$\|X\|_{\infty} = \max_{1 \leq i \leq n} |x_i|$$

En Python, on peut la calculer avec la fonction `numpy.linalg.norm(X, inf)`.

Définition 2.2 - norme de Manhattan

Pour $X \in \mathbb{R}^n$, on définit la norme de Manhattan par :

$$\|X\|_1 = \sum_{i=1}^n |x_i|$$

En Python, on peut la calculer avec la fonction `numpy.linalg.norm(X, 1)`.

Définition 2.3 - norme euclidienne

Pour $X \in \mathbb{R}^n$, on définit la norme euclidienne par :

$$\|X\|_2 = \left(\sum_{i=1}^n |x_i|^2 \right)^{1/2}$$

En Python, on peut la calculer avec la fonction `numpy.linalg.norm(X)`.

Définition 2.4 - norme subordonnée

Pour $A \in \mathcal{M}_n(\mathbb{R})$, on définit la norme subordonnée associée à une norme vectorielle $\|\cdot\|$ par :

$$\|A\| = \sup_{X \in \mathbb{R}^n, X \neq 0} \frac{\|AX\|}{\|X\|}$$

En Python, on peut la calculer avec la fonction `numpy.linalg.norm(A, ord)` où `ord` est l'ordre de la norme vectorielle associée (1, 2 ou `inf`).

Remarque 2.5 - sur la norme euclidienne d'une matrice

On a $\|A\|_2 = \sqrt{\rho(AA^\top)}$ où $\rho(M)$ est la valeur propre de M de plus grande valeur absolue. Cette norme est difficile à calculer en pratique. On préfère donc utiliser les normes $\|\cdot\|_1$ ou $\|\cdot\|_\infty$ qui sont plus rapides à calculer.

I.2 Erreur d'approximation et méthode d'estimation de cette erreur

Si A est inversible, on a :

$$b = Ax \quad \Rightarrow \quad \|b\| = \|Ax\| \leq \|A\| \|x\|$$

De même :

$$\begin{aligned} A \times (x + \delta x) &= b + \delta b \\ \Rightarrow A\delta x &= \delta b \\ \Rightarrow \|\delta b\| &= \|A\delta x\| \leq \|A\| \|\delta x\| \\ \Rightarrow \|b\| \|\delta b\| &\leq \|A\| \|x\| \|A^{-1}\| \|\delta x\| \\ \Rightarrow \underbrace{\frac{\|\delta x\|}{\|x\|}}_{\text{erreur relative sur la solution}} &\leq \underbrace{\|A\| \|A^{-1}\|}_{\text{cond}(A)} \underbrace{\frac{\|\delta b\|}{\|b\|}}_{\text{erreur que l'on mesure}} \end{aligned}$$

L'erreur est amplifiée d'un facteur appelé *conditionnement* de la matrice A , noté :

$$\text{cond}(A) = \|A\| \|A^{-1}\|$$

En Python, on le calcule avec `cond(A, ord)` (*cf.* plus haut).

Remarque 2.6 - *conditionnement d'une matrice symétrique définie positive*

Si $A \in S_n^+(\mathbb{R})$, alors $\text{cond}(A) = \frac{\lambda_{\max}}{\lambda_{\min}}$ où $\lambda_{\max}$ et $\lambda_{\min}$ sont les valeurs propres de A de plus grande et plus petite valeur absolue respectivement. On caractérise alors conditionnement comme la capacité d'une matrice à convertir les boules en ellipsoïdes :

- Plus le conditionnement est élevé, plus les boules sont transformées en ellipsoïdes allongés, et plus l'erreur est amplifiée.
- Plus le conditionnement est faible, plus les boules sont transformées en ellipsoïdes proches de cercles, et moins l'erreur est amplifiée.

Dans le cas général, le conditionnement d'une matrice A est égal au rapport entre les plus grandes et les plus petites valeurs singulières de A (généralisation des valeurs propres pour les matrices non carrées).

Définition 2.7 - *mauvais conditionnement*

On dit d'une matrice $A \in \mathcal{M}_n(\mathbb{R})$ qu'elle est *mal conditionnée* si son conditionnement est élevé :

- Si $\text{cond}(A) = 10^4$, alors A est mal conditionnée
- Si $\text{cond}(A) = 3$, alors A est bien conditionnée.

Moralement, pour résoudre $Ax = b$:

- On trouve $x + \delta x$
- On évalue le *résidu* $\|\delta b\| = \|b - A \times (x + \delta x)\|$

Remarque 2.8 - *mauvais conditionnement*

Les matrices obtenues en discrétisant des équations physiques sont généralement mal conditionnées.

I.3 Calculs de complexité

Les additions, soustractions et multiplication ont aujourd'hui des coûts temporels semblables, mais la division est plus complexe. On cherche à s'en passer le plus possible.

Exemple 2.9 - *complexité du produit d'une matrice avec un vecteur*

Soit $A \in \mathcal{M}_{n,p}(\mathbb{R})$ et $x \in \mathbb{R}^n$. Notons $y = Ax$ et $y = (y_1, \dots, y_n)^\top$. On a :

$$\forall i \in \llbracket 1, n \rrbracket, y_i = \sum_{j=1}^p a_{i,j} x_j$$

. Le calcul de chaque y_i demande donc $2p$ opérations, ainsi le calcul de y demande $2np$ opérations.

Exemple 2.10 - résolution d'un système linéaire triangulaire

Soit $T \in \mathcal{M}_{n,p}(\mathbb{R})$ et $b \in \mathbb{R}^n$. Calculons la complexité de la résolution de $Tx = b$. On prend T triangulaire inférieure. Ainsi :

$$Tx = b \Leftrightarrow \begin{cases} t_{1,1}x_1 = b_1 \\ t_{2,1}x_1 + t_{2,2}x_2 = b_2 \\ \vdots \\ t_{n,1}x_1 + t_{n,2}x_2 + \cdots + t_{n,n}x_n = b_n \end{cases}$$

$$\Leftrightarrow \begin{cases} x_1 = \frac{b_1}{t_{1,1}} \\ x_2 = \frac{b_2 - t_{2,1}x_1}{t_{2,2}} \\ \vdots \\ x_n = \frac{b_n - \sum_{j=1}^{n-1} t_{n,j}x_j}{t_{n,n}} \end{cases}$$

Les coûts opératoires pour chaque ligne vérifient le tableau suivant :

Ligne	additions	soustractions	multiplications	divisions
1	0	0	0	1
2	0	1	1	1
3	0	2	2	1
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	0	$n-1$	$n-1$	1
Somme	0	$\sum_{i=0}^{n-1} i$	$\sum_{i=0}^{n-1} i$	n

On a donc n^2 opérations au total.

I.4 Méthodes de résolution de $Ax = b$ **I.4.1 Par factorisation de A** **Exemple 2.11** - produit de matrices triangulaires

Si $A = LU$ avec L et U triangulaires, alors pour résoudre $Ax = b$, on résout successivement :

- $Ly = b$ (coût n^2)
- $Ux = y$ (coût n^2)

Le coût total est donc de $2n^2$ opérations.

I.5 Par méthode itérative

On cherche à construire une suite $(x^{(k)})_{k \in \mathbb{N}}$ telle que :

$$\lim_{k \rightarrow +\infty} x^{(k)} = x$$

où x est la solution de $Ax = b$. On arrête le processus itératif lorsque le *résidu relatif* $\frac{\|\delta b\|}{\|b\|} = \frac{\|b - Ax^{(k)}\|}{\|b\|}$ est inférieur à un seuil fixé ε :

$$\frac{\|x - x_k\|}{\|x\|} = \frac{\|\delta x\|}{\|x\|} \leq \text{Cond}(A)\varepsilon$$

Exemple 2.12 - de choix d' ε

Si on veut une erreur relative sur la solution inférieure à 10^{-6} et que le conditionnement de A est de 10^4 , on choisit :

$$\varepsilon = \frac{10^{-6}}{10^4} = 10^{-10}$$

Remarque 2.13 - avantages de cette méthode

Cette méthode présente plusieurs avantages :

- pas de nécessité de stocker la matrice A en mémoire
- pas de nécessité de calculer une factorisation de A qui pourrait présenter un phénomène de *remplissage* (beaucoup de zéros dans A deviennent des non-zéros dans la factorisation)

Remarque 2.14 - désavantages de cette méthode

Cette méthode présente aussi des inconvénients :

- le nombre d'itérations nécessaires pour atteindre la précision souhaitée peut être élevé et il augmente avec le conditionnement de A .
- le choix de la méthode itérative et de son paramétrage (ex : préconditionneur) peut être délicat.

Pour éviter ces inconvénients, on peut tenter de préconditionner la matrice A pour améliorer son conditionnement, ce qui revient à résoudre :

$$M^{-1}Ax = M^{-1}b$$

où M est un *préconditionneur* choisi pour que $M^{-1}A$ soit mieux conditionnée que A .

II Factorisation de Cholesky

Soit $A \in \mathcal{M}_n(\mathbb{R})$ une matrice symétrique définie positive. On cherche à factoriser A sous la forme :

$$A = LL^\top$$

où L est une matrice triangulaire inférieure.

Remarque 2.15 - *matrices symétriques définies positives usuelles*

Une matrice symétrique $A \in S_n(\mathbb{R})$ à diagonale strictement dominante :

$$\forall i \in \llbracket 1, n \rrbracket, a_{ii} > \sum_{j \neq i} |a_{ij}|$$

est définie positive. C'est un bon moyen de construire des matrices définies positives en pratique, ou de vérifier l'appartenance à S_n^+ .

Une telle matrice L , on a :

$$A = \begin{pmatrix} l_{11} & 0 & 0 & \cdots & 0 \\ l_{21} & l_{22} & 0 & \cdots & 0 \\ l_{31} & l_{32} & l_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & l_{n3} & \cdots & l_{nn} \end{pmatrix} \times \begin{pmatrix} l_{11} & l_{21} & l_{31} & \cdots & l_{n1} \\ 0 & l_{22} & l_{32} & \cdots & l_{n2} \\ 0 & 0 & l_{33} & \cdots & l_{n3} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & l_{nn} \end{pmatrix}$$

D'où :

$$A = \begin{pmatrix} l_{11}^2 & l_{11}l_{21} & l_{11}l_{31} & \cdots & l_{11}l_{n1} \\ l_{11}l_{21} & l_{21}^2 + l_{22}^2 & l_{21}l_{31} + l_{22}l_{32} & \cdots & l_{21}l_{n1} + l_{22}l_{n2} \\ l_{11}l_{31} & l_{21}l_{31} + l_{22}l_{32} & l_{31}^2 + l_{32}^2 + l_{33}^2 & \cdots & l_{31}l_{n1} + l_{32}l_{n2} + l_{33}l_{n3} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ l_{11}l_{n1} & l_{21}l_{n1} + l_{22}l_{n2} & l_{31}l_{n1} + l_{32}l_{n2} + l_{33}l_{n3} & \cdots & \sum_{k=1}^n l_{nk}^2 \end{pmatrix}$$

En comparant les coefficients de la première colonne, on obtient :

$$\begin{cases} a_{11} = l_{11}^2 \\ a_{21} = l_{11}l_{21} \\ a_{31} = l_{11}l_{31} \\ \vdots \\ a_{n1} = l_{11}l_{n1} \end{cases} \Rightarrow \begin{cases} l_{11} = \sqrt{a_{11}} \\ l_{i1} = \frac{a_{i1}}{l_{11}} \text{ pour } i \in \llbracket 2, n \rrbracket \end{cases}$$

Puis pour la deuxième colonne :

$$\begin{cases} a_{22} = l_{21}^2 + l_{22}^2 \\ a_{32} = l_{21}l_{31} + l_{22}l_{32} \\ \vdots \\ a_{n2} = l_{21}l_{n1} + l_{22}l_{n2} \end{cases} \Rightarrow \begin{cases} l_{22} = \sqrt{a_{22} - l_{21}^2} \\ l_{i2} = \frac{a_{i2} - l_{21}l_{i1}}{l_{22}} \text{ pour } i \in \llbracket 3, n \rrbracket \end{cases}$$

Puis pour la i -ème colonne :

$$\begin{cases} a_{ii} = \sum_{k=1}^i l_{ik}^2 \\ a_{(i+1)i} = \sum_{k=1}^i l_{(i+1)k} l_{ik} \\ \vdots \\ a_{ni} = \sum_{k=1}^i l_{nk} l_{ik} \end{cases} \Rightarrow \begin{cases} l_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2} \\ l_{ji} = \frac{a_{ji} - \sum_{k=1}^{i-1} l_{jk} l_{ik}}{l_{ii}} \end{cases} \text{ pour } j > i$$

Exemple 2.16 - factorisation de Cholesky

Tentons de calculer la factorisation de Cholesky de la matrice :

$$A = \begin{pmatrix} 4 & -2 & -4 \\ -2 & 10 & 5 \\ -4 & 5 & 6 \end{pmatrix}$$

En travaillant dans l'ordre croissant des colonnes, on trouve :

$$L = \begin{pmatrix} 2 & 0 & 0 \\ -1 & 3 & 0 \\ -2 & 1 & 1 \end{pmatrix}$$

III Cas où A n'est que symétrique

Lorsque A est symétrique, mais pas définie positive, on ne peut pas utiliser la décomposition de Crout :

$$A = LDL^T$$

où L est triangulaire inférieure à diagonale par blocs de taille 1 ou 2, unitaire (remplie de 1), et D est diagonale.

Remarque 2.17 - sur la complexité

La complexité de la factorisation de Crout est également de $O(\frac{n^3}{3})$, et la résolution du système linéaire à partir de cette factorisation est de $O(n^2)$, comme pour la factorisation de Cholesky. On néglige dans ces calculs la complexité du calcul des racines carrés, qui est de $O(1)$.

IV Cas où A n'est pas symétrique

Lorsque A n'est pas symétrique, on peut utiliser la décomposition de LU :

$$A = LU$$

où L est triangulaire inférieure à diagonale unitaire (remplie de 1) et U est triangulaire supérieure. On applique un algorithme basé sur le pivot de Gauss pour calculer cette factorisation, qui a une complexité de $O(\frac{2n^3}{3})$. La résolution du système linéaire à partir de cette factorisation est de $O(n^2)$.

```

1 | U = A
2 | L = I_n
3 | pour i = 1 à n:
4 |     // On veut que la colonne i de U soit nulle à partir de la ligne i+
5 |     pour j = i+1 à n:
6 |         alpha = U[j][i] / U[i][i]
7 |         L[j][i] = alpha
8 |         pour k = i à n:
9 |             U[j][k] = U[j][k] - alpha * U[i][k]
```

Exemple 2.18 - factorisation LU

Tentons de calculer la factorisation LU de la matrice :

$$A = \begin{pmatrix} 2 & -1 & 3 \\ -4 & -1 & -2 \\ 6 & -9 & 16 \end{pmatrix}$$

V Implémentation de $\mathcal{M}_{n,p}(\mathbb{R})$

Dans cette section nous étudions différentes méthodes de stockage des matrices en mémoire.

V.1 Le plus fréquent : CSR

La méthode *Compressed Sparse Row (CSR)* consiste à stocker uniquement les éléments non nuls de la matrice, ainsi que leurs indices de ligne et de colonne. On utilise trois tableaux :

1. **data** : tableau des valeurs non nulles
2. **indices** : tableau des indices de colonnes des valeurs non nulles
3. **ptr** : tableau des indices dans **data** où commence chaque ligne

V.2 Partie Python

On utilise `L = cholesky(A).T` (dans `numpy.linalg`) puis `y = solve_triangular(L, b, lower=True)` (dans `scipy.linalg`) pour résoudre $Ly = b$. Ensuite on utilise `x = solve_triangular(L.T, y, lower=False)` pour résoudre $L^T x = y$.